

Design Metrics

1	Simulation time in ns (6tests)	6,298,1520 ns
2	Simulation cycle count	12,596,304 cycles
3	Synthesized clock period	4.85 ns
4	Comb Area	78179um ²
5	Non Comb Area	124,633um ²
6	Total Area	202,813um ²
7	Setup Slack	0.0008 ns
8	Hold Slack	0.0239 ns
9	Synthesis time	694 seconds
10	Performance/area	$8.07 \times 10^{-14} / \text{ns} \cdot \text{um}^2$

$$12,596,304 \times 4.85 = 6.1092\text{E}7 \times 202813 = 1.2390\text{E}13$$

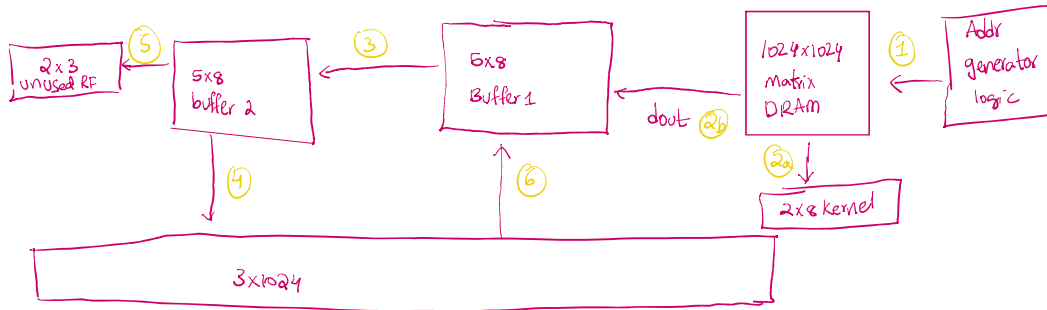
$$1/1.2390\text{E}13 = 8.07 \times 10^{-14}$$

$$12596268 \times 6 = 75577608 \times 199006 = 1\text{E}13$$

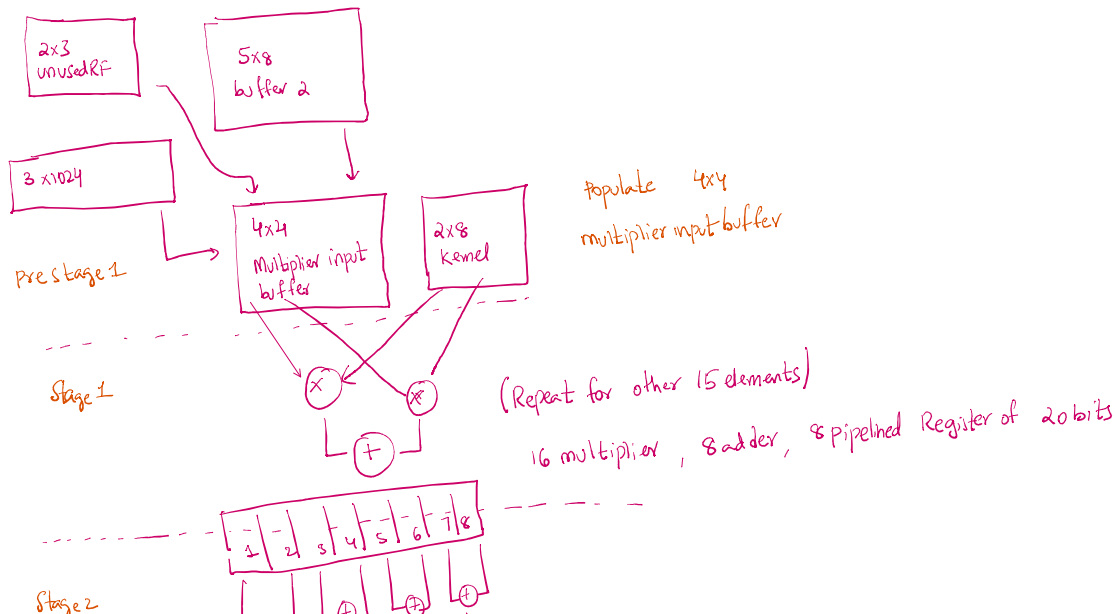
$$6.64 \times 10^{-14}$$

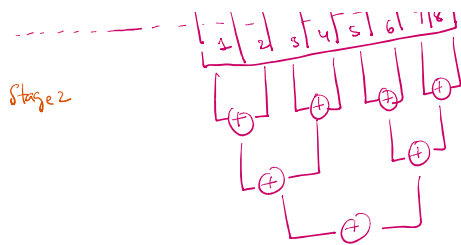
High Level Data Flow strategy to finish the simulation in 1.001 times of 1024*1024 cycles

1. Addr generator logic generates address and read command to preloaded DRAM.
2. Dout from dram fills 16 bytes of kernel and once kernel is downloaded, dout is redirected to 5*8 buffer 1.
3. When buffer 1 is filled, copy buffer 1 contents to buffer 2. Start Sliding in N pattern. N pattern will completely utilize the first two rows of buffer 2.
4. Copy unused elements to 3*1024.
5. Copy unused elements to 2*3 unused RF
6. When address generator issues reads for only 2 rows of a matrix, copy other 3 rows from 3*1024



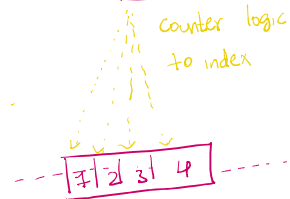
Pipelined MAC





1 pipelined Register of 20bits

Stage 3



greater $> 0 \Rightarrow$ stays same
 Less $< -4 \Rightarrow$ divide by 4
 else $\Rightarrow 0$

Leaky Relu, 4 Register of 20bits

mux logic to add $1+2+3+4$ (General case).

Stage 4

$1+3$ (Last Row zero padding)
 $1+2$ (Last column zero padding)
 1 (Last Row zero padding + Last column zero padding)

↓
 Avg pool

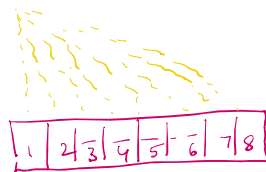
mux output $> 4 \Rightarrow$ divide by 4

$< -4 \Rightarrow$ divide by 4

else $\Rightarrow 0$

↓
 Clipping

Counter logic
 to index

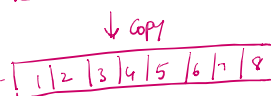


Avg pool output $> 127 \Rightarrow 127$

$< -128 \Rightarrow -128$

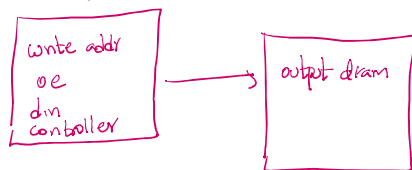
else $\Rightarrow$ Avg pool output

Stage 5



↓ Copy

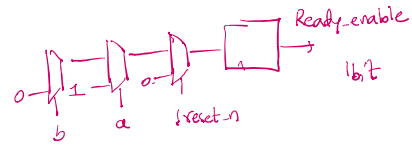
Stage 6



Micro Architecture of the above strategy with system verilog code in the same order

```
//Handshake
always@(posedge clk) begin
  if(!reset_n)begin
    ready_enable <= 0;
  end
  else if(ready && start) begin
    ready_enable <= 1;
  end
  // else if (matrix_count == 1)
  //   ready_enable <= 0;
  // else if (base_addr == (ROWS*COLUMN))
  //   ready_enable <= 0;
  else if (!addr_1 == END)
    ready_enable <= 0;
  end
```

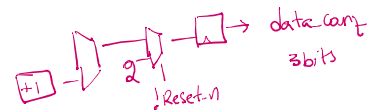
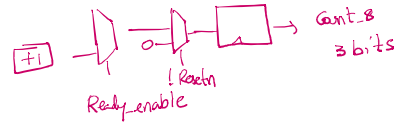
Handshake
Start/stop condition of
Simulation



Ready_enable → Ready

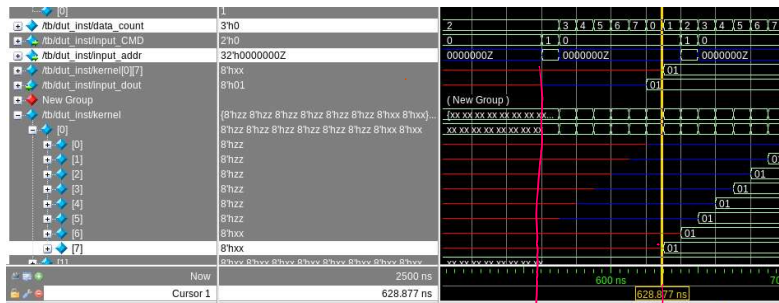
```
//count_8
always@(posedge clk) begin
  if(!reset_n) begin
    count_8 <= 0;
    data_count <= 2;
  end
  else if(ready_enable) begin
    count_8 <= count_8 + 1;
    data_count <= data_count + 1;
  end
end
```

Pre stage 1
Addr generator logic
data capture logic



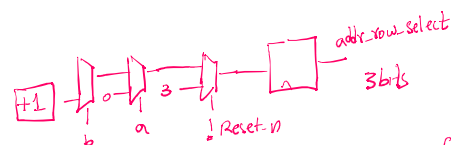
Count_8 — 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
 ↑ ↑ ↑
 cmd cmd cmd
 addr addr addr

data_count — 2 3 4 5 6 7 0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7
 Latency data capture 1st 8 bytes data capture 2nd 8 bytes



Addr generator logic
data capture logic

```
//addr_row_select
always@(posedge clk)begin
  if(!reset_n)begin
    addr_row_select <= 3;
    kernel_full <= 0;
  end
  else if(addr_row_select == row_5_2 && count_8 == 5) begin
    addr_row_select <= 0;
    kernel_full <= 1;
  end
  else if(count_8 == 5)
    addr_row_select <= addr_row_select + 1;
end
```



Kernel
addr_row_select-3 4 | matrix first Rows till end | matrix from 6th Row
 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4
kernel_full 0 0 1 1
data_row_select 3 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 3 4

data capture logic

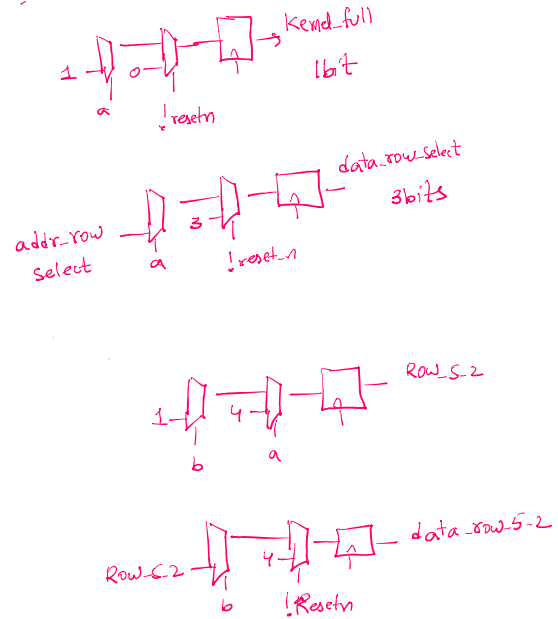
data capture logic

```
always@(posedge clk)begin
  if(!reset_n)begin
    data_row_select <= 3;
  end
  else if(data_count == 7)
    data_row_select <= addr_row_select;
  end
end
```

```
always@(posedge clk) begin
  if(!reset_n)begin
    row_5_2 <= 4;
  end
  else if(compare_addr > 0)begin
    row_5_2 <= 1;
  end
end

always@(posedge clk)begin
  if(!reset_n) begin
    data_row_5_2 <= 4;
  end
  else if(data_row_select == 4 && data_count == 6)
    data_row_5_2 <= row_5_2;
  end
end
```

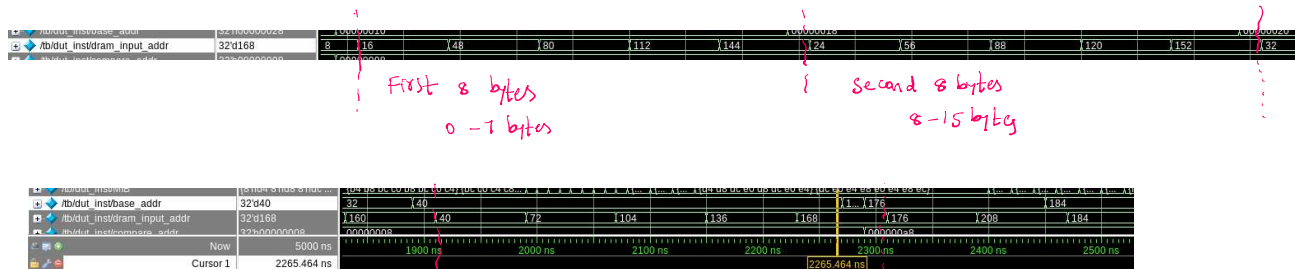
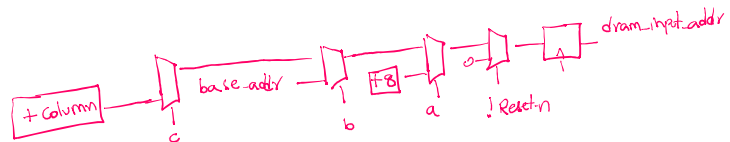
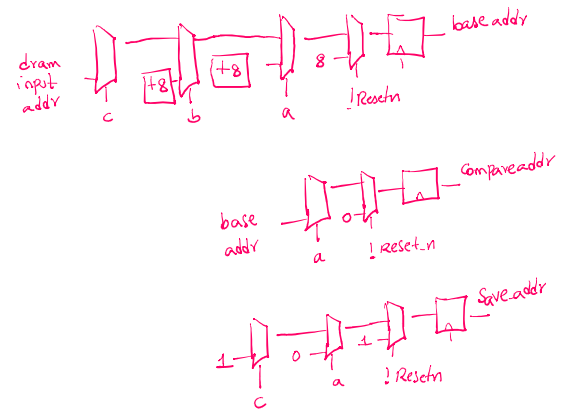
Limits data_row_select,
addr_row_select
max value



Addr generator logic

```
//address
always@(posedge clk)begin
  if(!reset_n) begin
    base_addr <= 0;
    save_addr <= 1;
    compare_addr <= 0;
  end
  else if(count_8 == 4 && addr_row_select == row_5_2 && save_addr)begin
    compare_addr <= base_addr;
    base_addr <= base_addr + 8;
    save_addr <= 0;
  end
  else if(count_8 == 4 && addr_row_select == row_5_2)begin
    base_addr <= base_addr + 8;
  end
  else if(count_8 == 2 && addr_row_select == row_5_2 && row_shift && !save_addr)begin
    base_addr <= dram_input_addr;
    save_addr <= 1;
  end
end

always@(posedge clk)begin
  if(!reset_n)
    dram_input_addr <= 0;
  else if(count_8 == 6 && ready_enable && !kernel_full)
    dram_input_addr <= dram_input_addr + 8;
  else if(count_8 == 6 && addr_row_select == 0 && ready_enable) //addr_row_select is incremented before count_8
    dram_input_addr <= base_addr;
  else if(count_8 == 6 && addr_row_select != 0 && ready_enable)
    dram_input_addr <= dram_input_addr + COLUMN;
  end
end
```



Input buffer size 5*8 = 40 Flipflop. Each flipflop has its own Write enable and Data_in.
Two sources of data_in, one byte from input_dout or 8 bytes from 3*1024 matrix buffer.
Data_row_select, 7-data_count will enable one flip flop.

We are copying 1 byte of data or 1+24 bytes of data at any given point.

24 - 31 bytes

First 8 bytes

0-7 bytes

n. and matrix end

0-7 bytes

Reached matrix end

The diagram illustrates a 2D array structure, likely a memory array or a data structure in a hardware design. It consists of a grid of cells, with a horizontal axis labeled "column select" and a vertical axis labeled "row select".

On the left side, there are two input buffers, each labeled "input buffer-1". Each buffer has a "double word line" and a "double word line" (labeled "a"). The buffers are connected to the grid via "double word line" and "double word line" (labeled "a").

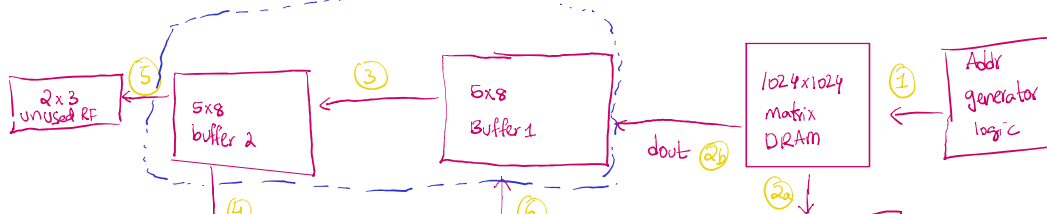
On the right side, there are two output buffers, each labeled "output buffer-1". Each buffer has a "double word line" and a "double word line" (labeled "a"). The buffers are connected to the grid via "double word line" and "double word line" (labeled "a").

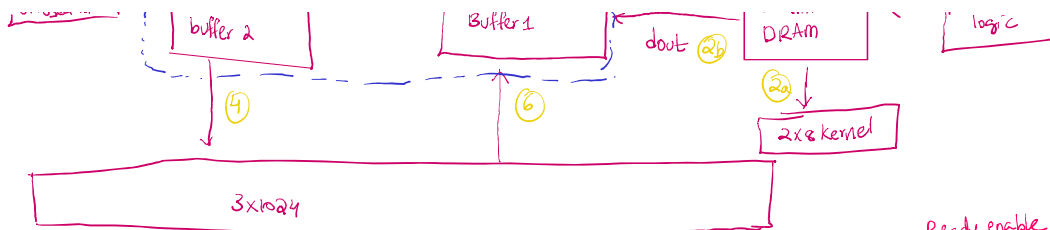
At the bottom, there is a "Ready-enable" signal, which is connected to the "double word line" and "double word line" (labeled "a") of the output buffers.

At the top, there is a "data row select" signal, which is connected to the "double word line" and "double word line" (labeled "a") of the input buffers.

Hand-drawn block diagram illustrating a 2D convolution architecture. The components and data flow are as follows:

- Address generator logic** (top right) provides address **(1)** to the **1024x1024 Matrix DRAM**.
- The **1024x1024 Matrix DRAM** outputs data **(2)** to the **2x8 kernel**.
- The **2x8 kernel** outputs data **(3)** to **5x8 Buffer 1**.
- 5x8 Buffer 1** outputs data **(4)** to **5x8 Buffer 2**.
- 5x8 Buffer 2** outputs data **(5)** to the **2x3 unused RF**.
- The **2x3 unused RF** outputs data **(6)** back to **5x8 Buffer 1**.
- A large block labeled **3x1024** is shown at the bottom, connected to the **5x8 Buffer 1** and **5x8 Buffer 2**.
- A dashed blue line encloses the **1024x1024 Matrix DRAM**, **2x8 kernel**, and **5x8 Buffer 1** components.

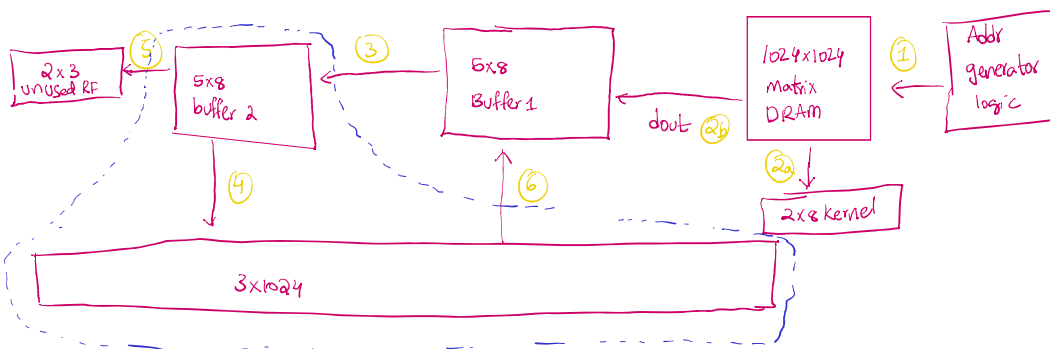
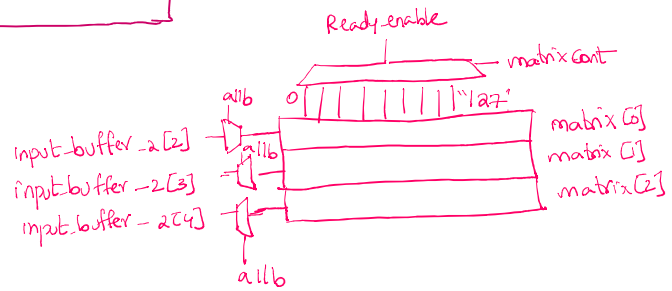




```

always@(posedge clk)begin
  if(!reset_n || matrix_count == MAXREAD) begin
    matrix_count <= 0;
  end
  if(copy_main)begin a
    matrix[0][matrix_count] <= input_buffer_2[2];
    matrix[1][matrix_count] <= input_buffer_2[3];
    matrix[2][matrix_count] <= input_buffer_2[4];
    matrix_count <= matrix_count + 1;
  end
  else if(copy_2)begin b
    matrix[0][matrix_count] <= input_buffer_2[2];
    matrix[1][matrix_count] <= input_buffer_2[3];
    matrix[2][matrix_count] <= input_buffer_2[4];
    matrix_count <= matrix_count + 1;
  end
end
end

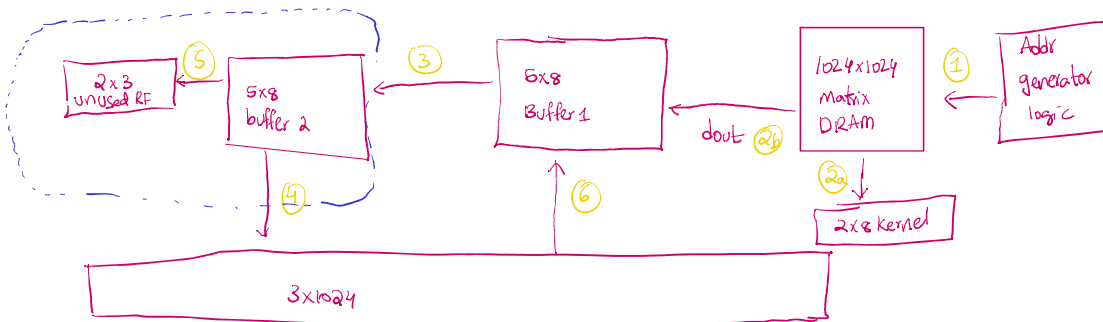
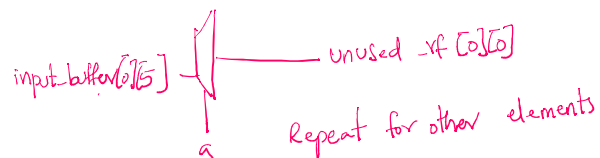
```



```

always@(posedge clk)begin
  if(slide_counter==7) begin a
    unused_rf[0][0] <= input_buffer_2[0][5];
    unused_rf[0][1] <= input_buffer_2[0][6];
    unused_rf[0][2] <= input_buffer_2[0][7];
    unused_rf[1][0] <= input_buffer_2[1][5];
    unused_rf[1][1] <= input_buffer_2[1][6];
    unused_rf[1][2] <= input_buffer_2[1][7];
  end
end
end

```



```

always@(posedge clk)begin
  if(slide_counter==7) begin
    unused_rf[0][0] <= input_buffer_2[0][5];
    unused_rf[0][1] <= input_buffer_2[0][6];
    unused_rf[0][2] <= input_buffer_2[0][7];
    unused_rf[1][0] <= input_buffer_2[1][5];
    unused_rf[1][1] <= input_buffer_2[1][6];
    unused_rf[1][2] <= input_buffer_2[1][7];
  end
end
end

```

Prestage 1

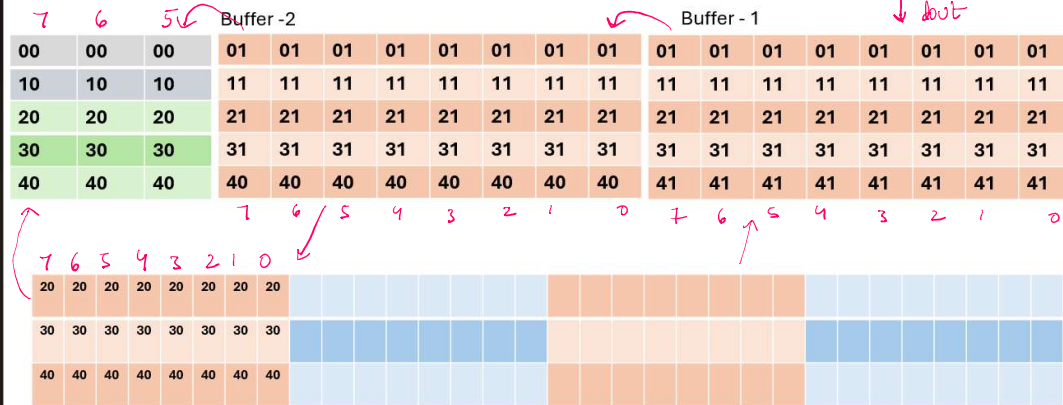
Input DRAM

```

unused_rf[1][1] <= input_buffer_2[1][0];
unused_rf[1][2] <= input_buffer_2[1][7];
end
end

```

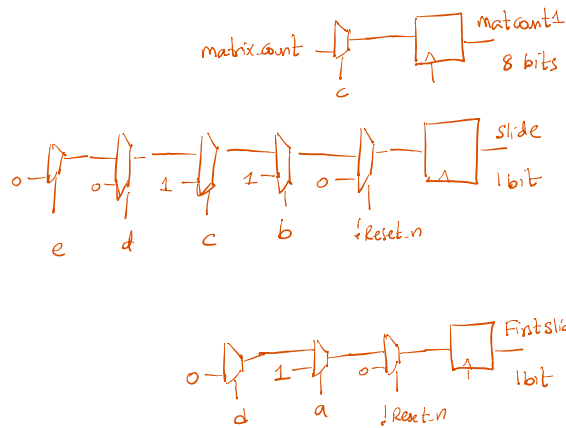
Input DRAM



```

//convolution slide control logic generation
//first slide flag control logic
always@(posedge clk)begin
  if(!reset_n)begin
    slide<=0;
    first_slide <=1;
  end
  else if(matcount==MAXREAD-1 && slide_counter==10) begin
    first_slide <=1;
  end
  else if(data_count==7 && data_row_select==1 && row2 && base_addr==24)begin
    slide<=1;
    matcount1 <= matrix_count;
  end
  else if(data_count==7 && data_row_select==4 && (row2 && base_addr==24)begin
    slide<=1;
    matcount1 <= matrix_count;
  end
  else if(slide_counter==9 && first_slide)begin
    first_slide <=0;
    slide<=0;
  end
  else if(slide_counter==15)
    slide<=0;
end

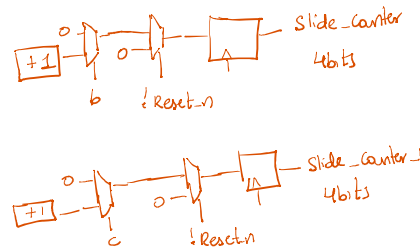
```



```

always@(posedge clk)begin
  if(slide_counter==7)
    matcount <= matcount1;
end
//first slide control logic
always@(posedge clk)begin
  if(!reset_n)begin
    slide_counter <=0;
  end
  else if(slide)
    slide_counter <= slide_counter + 1;
  else
    slide_counter <=0;
end
always@(posedge clk)begin
  if(!reset_n)begin
    slide_counter_1 <=0;
  end
  else if(slide_del_1)
    slide_counter_1 <= slide_counter_1 + 1;
  else
    slide_counter_1 <=0;
end

```



Hard coded. wires connected between macinput and inputbuffer

```

always@(posedge clk) begin
  if(slide) begin
    if(!first_slide) begin
      macinput[0][0] <= input_buffer_2[0][0]; macinput[0][1] <= input_buffer_2[0][1]; macinput[0][2] <= input_buffer_2[0][2]; macinput[0][3] <= input_buffer_2[0][3];
      macinput[0][4] <= input_buffer_2[1][0]; macinput[0][5] <= input_buffer_2[1][1]; macinput[0][6] <= input_buffer_2[1][2]; macinput[0][7] <= input_buffer_2[1][3];
      macinput[1][0] <= input_buffer_2[2][0]; macinput[1][1] <= input_buffer_2[2][1]; macinput[1][2] <= input_buffer_2[2][2]; macinput[1][3] <= input_buffer_2[2][3];
      macinput[1][4] <= input_buffer_2[3][0]; macinput[1][5] <= input_buffer_2[3][1]; macinput[1][6] <= input_buffer_2[3][2]; macinput[1][7] <= input_buffer_2[3][3];
    end
    else begin
      macinput[0][0] <= unused_rf[0][0]; macinput[0][1] <= unused_rf[0][1]; macinput[0][2] <= unused_rf[0][2]; macinput[0][3] <= input_buffer_2[0][0];
      macinput[0][4] <= unused_rf[1][0]; macinput[0][5] <= unused_rf[1][1]; macinput[0][6] <= unused_rf[1][2]; macinput[0][7] <= input_buffer_2[1][0];
      macinput[1][0] <= matrix[0][matcount][5]; macinput[1][1] <= matrix[0][matcount][6]; macinput[1][2] <= matrix[0][matcount][7]; macinput[1][3] <= input_buffer_2[2][0];
      macinput[1][4] <= matrix[1][matcount][5]; macinput[1][5] <= matrix[1][matcount][6]; macinput[1][6] <= matrix[1][matcount][7]; macinput[1][7] <= input_buffer_2[3][0];
    end
  end
end

```

always@(posedge clk)begin

two class 1 (slide) - macinput[0][3] is populated

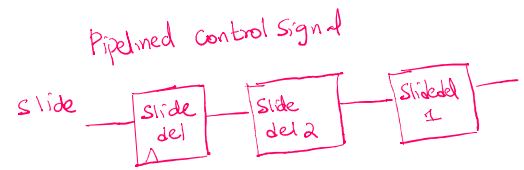
Pipelined control signal

```

always@(posedge clk)begin
    slide_del <= slide;
    slide_del_2 <= slide_del;
    slide_del_1 <= slide_del_2;
end

```

Pre stage 1 (slide) — macinput[0:3] is populated
 stage 1 (slide-del) — multiplier and adder
 stage 2 (slide-del_2) — only adder
 stage 3 (slide-del_1) — leakyRelu



Prestage complete

```

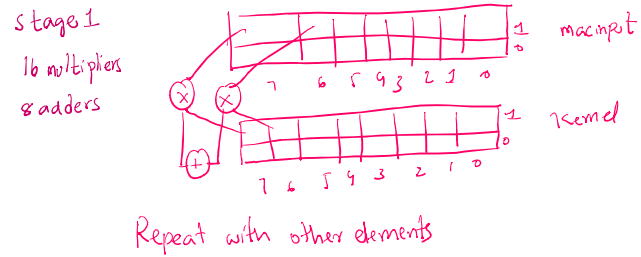
always@(posedge clk)begin
    if(slide_del) begin
        // mac_final = (mac1+mac2)+(mac3+mac4)+(mac5+mac6+mac7+mac8);
        row1_res <= macinput[0][0] * kernel[0][0] + macinput[0][1] * kernel[0][1];
        row5_res <= macinput[0][2] * kernel[0][2] + macinput[0][3] * kernel[0][3];

        row2_res <= macinput[0][4] * kernel[0][4] + macinput[0][5] * kernel[0][5];
        row6_res <= macinput[0][6] * kernel[0][6] + macinput[0][7] * kernel[0][7];

        row3_res <= macinput[1][0] * kernel[1][0] + macinput[1][1] * kernel[1][1];
        row7_res <= macinput[1][2] * kernel[1][2] + macinput[1][3] * kernel[1][3];

        row4_res <= macinput[1][4] * kernel[1][4] + macinput[1][5] * kernel[1][5];
        row8_res <= macinput[1][6] * kernel[1][6] + macinput[1][7] * kernel[1][7];
        //mac_final = ((mac1+mac2)+(mac3+mac4))+((mac5+mac6)+(mac7+mac8));
    end
end

```



stage 1 complete

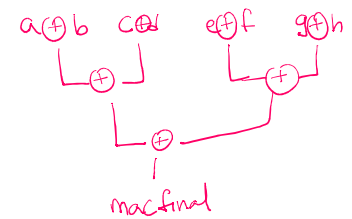
```

always@(posedge clk)begin
    if(slide_del_2)
        mac_final <= ((row1_res+row2_res)+(row3_res+row4_res))+((row5_res+row6_res)+(row7_res+row8_res));
end

```

a b c d e f g h

Stage 2
 8 adders
 Convolved output



stage 2 complete

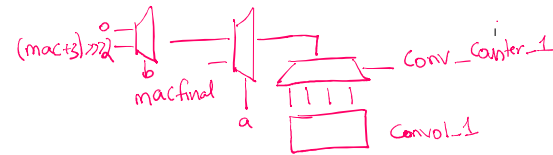
```

always@(posedge clk)begin
    if(slide_counter_1)
        convol_1[conv_counter_1] <= mac_final > 0 ? mac_final : mac_final < -4 ? (mac_final+3)>>2 : 0;
end

```

a b

Stage 3
 Leaky Relu



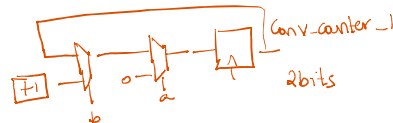
```

always@(posedge clk)begin
    if(reset_n || (slide_counter_1 == 15 && first_slide))begin
        conv_counter_1 <= 0;
    end
    else if(slide_del_1)begin
        conv_counter_1 <= conv_counter_1 + 1;
    end
end

```

a b

Stage 3



```

always@(posedge clk)begin
    if(slide_del_1)
        if(conv_counter_1==3)begin
            conv_full_1<=1;
        end
        else if(slide_counter_1 == 15 && first_slide)begin
            conv_full_1<=1;
        end
        else begin
            conv_full_1<=0;
        end
end

```

a b c d

Stage 3



stage 3 complete

System verilog unique feature

```

always_comb begin
    clip_pool_1 = 0;

    if(conv_full_1)begin
        conv_out_1 = (convol_1[0] + convol_1[1] + convol_1[2] + convol_1[3] );

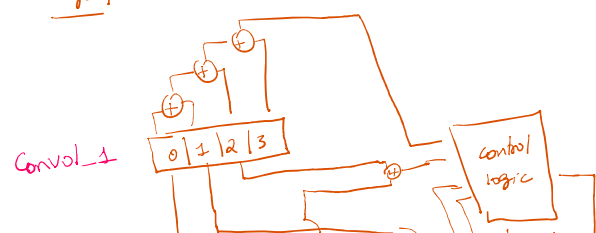
        if(data_complete && pool_counter_1==6)
            conv_out_1 = (convol_1[0]);
        else if(pad_zero_reg)
            conv_out_1 = (convol_1[0] + convol_1[2]);
        else if(slide_counter_1 == 0 && first_slide)
            conv_out_1 = (convol_1[0] + convol_1[1]);

        avg_pool_1 = (conv_out_1 >= 4 ? conv_out_1>>2 : conv_out_1 < -4 ? (conv_out_1+3)>>2 : 0) ;
        clip_pool_1 = avg_pool_1 > 127 ? 127 : avg_pool_1 < -128 ? -128 : avg_pool_1;
    end
end

```

a b c d

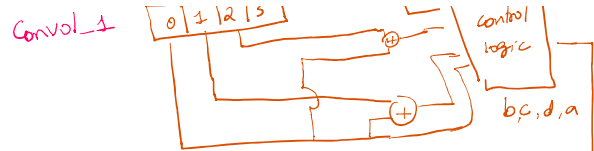
Stage 4



```

else if (slide_counter_1 == 0 && first_slide) d
  conv_out_1 = (convol_1[0] + convol_1[1]);
  avg_pool_1 = (conv_out_1 >= 4 ? conv_out_1 >> 2 : conv_out_1 <= -4 ? (conv_out_1+3) >> 2 : 0);
  clip_pool_1 = avg_pool_1 > 127 ? 127 : avg_pool_1 < -128 ? -128 : avg_pool_1;
end
end

```



```

always@(posedge clk)begin
  if (conv_full_1 && (slide_counter_1 == 0 && first_slide)) begin
    clip_pool_buffer_1[pool_counter_1] <= clip_pool_1;
    clip_pool_buffer_1[7] <= 8'h00;
  end
  else if (conv_full_1) begin
    clip_pool_buffer_1[pool_counter_1] <= clip_pool_1;
  end
end

```



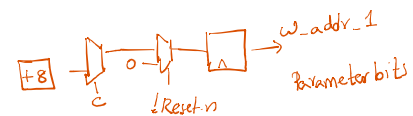
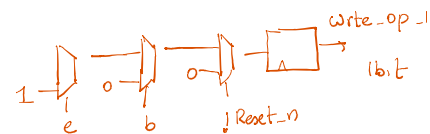
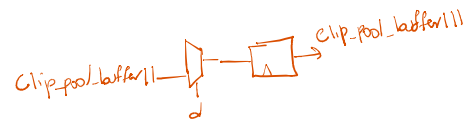
stage 4 complete

```

always@(posedge clk)begin
  if (!reset_n)begin
    write_op_1 <= 0;
    w_addr_1 <= 0;
  end
  else if (pool_index_write_1 == 0) b
    write_op_1 <= 0;

  else if (pool_index_write_1 == 1) c
    w_addr_1 <= w_addr_1 + 8;
  else if (pool_count_7 && pool_counter_1 == 0 && conv_counter_1 == 1) d
    clip_pool_buffer_1[1] <= clip_pool_buffer_1[1];
  else if (pool_count_7 && pool_counter_1 == 1 && conv_counter_1 == 1) begin e
    write_op_1 <= 1;
  end
end

```

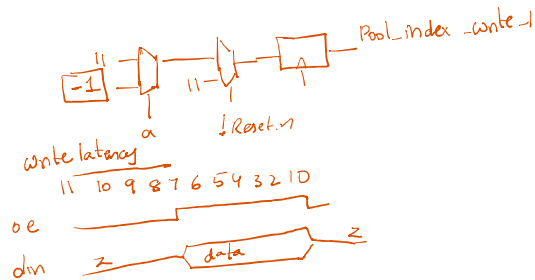


stage 5 complete

```

always@(posedge clk)begin
  if (!reset_n)
    pool_index_write_1 <= 11;
  else if (write_op_1)begin a
    pool_index_write_1 <= pool_index_write_1 - 1;
  end
  else
    pool_index_write_1 <= 11;
end

```



stage 6 complete

Design Metrics

- Simulation time in ns : 62,981,520 ns
- Simulation cycle count : 12,596,304 cycles
- Synthesized clock period : 4.85 ns
- Comb Area : 78179um²
- Non Comb Area : 124633um²
- Total Area : 202813um²
- Setup Slack : 0.0008 ns
- Hold Slack : 0.0239 ns
- Synthesis time : 694 seconds

Pipelines design formula - $1024 \times 1024 \times 1.25 \times 2 \times 6$ (multiplied by 2 to offset TB time calculation)

= 15,728,640 cycles
Completed in less than the given time.

```
csomana@grendel51:projectFall2025/build $ cat synth/reports/timing_max_slow_holdfixed_tut1.rpt | tail -15
clock clk (rise edge)          -> 4.8500    4.8500
clock network delay (ideal)     0.0000    4.8500
clock uncertainty               -0.0500    4.8000
row1_res_reg[15]/CK (DFF_X2)   0.0000    4.8000 r
library setup time             -0.2679    4.5321
data required time              -----
data required time              4.5321
data arrival time              -4.5313
slack (MET)                    -> 0.0008
```

```
1
csomana@grendel51:projectFall2025/build $
```

```
csomana@grendel51:projectFall2025/build $ cat synth/reports/timing_max_slow_holdfixed_tut1.rpt | tail -10
library setup time             -0.2679    4.5321
data required time              -----
data required time              4.5321
data arrival time              -4.5313
slack (MET)                    -> 0.0008
```

```
1
csomana@grendel51:projectFall2025/build $
```

```
NangateOpenCellLibrary_PDKv1_2_v2008_10_slow_nldm (File: /mnt/coe/workspace/ece/ece720-common/tech/nangate/NangateOpenCellLibrary_PDKv1_2_v2008_10/liberty/520/NangateOpenCellLibrary_PDKv1_2_v2008_10_slow_nldm.db)
Number of ports:                192
Number of nets:                 113326
Number of cells:                87752
Number of combinational cells:  61629
Number of sequential cells:     26031
Number of macros/black boxes:   0
Number of buf/inv:              2961
Number of references:            71
Combinational area:             78179.530021
Buf/Inv area:                   1763.924003
Noncombinational area:          124633.770780
Macro/Black Box area:           0.000000
Net Interconnect area:          undefined (No wire load specified)
Total cell area:                -> 202813.300801
Total area:                     undefined
1
Memory usage for this session 629 Mbytes.
Memory usage for this session including child processes 1089 Mbytes.
CPU usage for this session 674 seconds ( 0.19 hours ).
Elapsed time for this session 694 seconds ( 0.19 hours ).
Thank you...
[100%] Built target synth
csomana@grendel51:projectFall2025/build $
```

```
# Importing memory file: /mnt/ncsdrive/c/csomana/icnn/projectFall2025/inputs/inputs.dat
# INFO:LVD: Test: Passed
# INFO[TB]: Total number of cases : 6
# INFO[TB]: Total number of passes : 6
# INFO[TB]: Final Results : 100.00
# INFO[TB]: Final Time Result : 62981520 ns
# INFO[TB]: Final Cycle Result : 12596304 cycles
# [TB] done
# ** Note: $finish : /mnt/ncsdrive/c/csomana/icnn/projectFall2025/srcs/tb/tb.sv(318)
# Time: 62981520 ns Iteration: 1 Instance: /tb
# End time: 02:18:46 on Nov 20, 2025, Elapsed time: 0:01:04
# Errors: 0, Warnings: 1
[100%] Built target vsim-dut-c
csomana@grendel51:projectFall2025/build $
csomana@grendel51:projectFall2025/build $
csomana@grendel51:projectFall2025/build $ cat synth/reports/timing_min_fast_holdcheck_tut1.rpt | tail -10
library hold time              0.0000    0.0500
data required time              -----
data required time              0.0500
-----
grendel51.ece.nyu.edu 22% 32.02 GB / 503.05 GB 0.04 Mb/s 0.03 Mb/s 2 days Inturi hbsu bhithoma2 csclark4 (x2) bjdome
```